# Une image divisée en quatre

Paralléliser le traitement à l’intérieur d’une image puis la reconstruire.

programme_3_quadrants_paralleles.py

## Les lignes à comprendre

```
morceaux = [image.crop(zone) for zone in zones]
gris_morceaux = pool.map(gris, morceaux, chunksize=1)
finale = Image.new("L", image.size)
for zone, morceau in zip(zones, gris_morceaux):
    finale.paste(morceau, zone[:2])
```

**1. Calculer les zones.** quadrants() coupe à largeur // 2 et hauteur // 2. Les zones couvrent toute la photo, même lorsque ses dimensions sont impaires.

**2. Convertir les quatre morceaux.** Le processus principal utilise crop() pour extraire les quadrants. Le pool distribue les morceaux et chacun est converti avec image.convert('L').

**3. Recoller et contrôler.** map() conserve l’ordre de départ. paste() remet chaque morceau à son coin d’origine. Le programme vérifie le dernier lot et sauvegarde l’image reconstituée et les quatre morceaux.

## Mesurer sur le Raspberry Pi

Dans Spyder, ouvre le script à côté de **image1.jpg** et appuie sur **F5**. Entre **DÉBUT** et **FIN**, relève la puissance moyenne au wattmètre. La photo est répétée pendant au moins 15 s. Lecture, écriture et démarrage du pool sont hors mesure.

**Temps moyen par image (s)** = durée / nombre d’images.
**Énergie par image (J)** = puissance moyenne (W) × temps moyen par image (s).

| Puissance moyenne (W) | Temps par image (s) | Énergie par image (J) |
|---|---|---|
| À mesurer | À relever dans Spyder | À calculer |

**Pour la comparaison :** Une image réassemblée compte pour une image traitée. Le chronomètre inclut le découpage, les transferts et le réassemblage.